DLLM: Discrete Edit-Based Diffusion Language Modelling for Prompt-Response Generation

Technical report – non-autoregressive, edit-based text generation via iterative refinement on a structured canvas

1 Abstract

Autoregressive language models (AR LMs) currently dominate natural language generation. They phrase every task as a continuation problem: the model conditions on the tokens written so far and predicts a probability distribution over the next token, one step at a time, left to right. This *autocomplete* paradigm has proven extraordinarily effective for open-ended text and instruction following, and it benefits from a simple, scalable training objective (causal next-token prediction). Yet it carries structural costs: decoding is sequential, so latency grows linearly with output length; information flows only from left to right, so the model can never consult the future context it is about to write; once a token is emitted it cannot be revised, and small early errors propagate and compound; and the length of the output is rigidly coupled to the number of decoding steps.

Diffusion language models (DLMs) offer a fundamentally different formulation. Instead of generating one token at a time, a DLM starts from a fully corrupted sequence – typically a canvas of mask tokens or noise – and iteratively denoises the entire sequence in parallel, refining all positions simultaneously over a small number of steps. Because every position is conditioned bidirectionally on the whole canvas at each step, generation is fully parallel per step, and errors can be corrected across iterations rather than compounded. However, most discrete diffusion LMs inherit a serious limitation: the inference procedure is defined on a fixed-length masked canvas. The number of output tokens is decided before generation begins and never changes, which makes it difficult for the model to produce a short answer to one prompt and a long one to another.

This report describes a discrete diffusion language model that is built around the *prompt-to-output* (question-answer, dialogue) paradigm rather than the autocomplete paradigm. Given a prompt, the model initializes a structured canvas in which the prompt is preserved verbatim while a region of interleaved structural mask tokens provides output capacity. Generation proceeds by applying a small vocabulary of discrete edit operations – **KEEP, DELETE, REPLACE, INSERT, EXPAND** – across multiple refinement steps. A *tagger* head predicts which operation each position needs, and a *generator* head proposes the actual vocabulary tokens where content must be produced or revised. Training is supervised by a forward corruption process that constructs noisy canvases at a continuum of noise levels and derives, for every position, the exact edit operation needed to move the canvas back toward the clean target. The length of the output is therefore not a fixed parameter but a decision the model makes dynamically, by growing the canvas (EXPAND, INSERT) or pruning it (DELETE) at every step, conditioned on the prompt and the current state of the canvas.

2 Literature Survey

Two lines of work directly motivate this architecture: length-control mechanisms for diffusion LMs, and edit-based discrete diffusion for conditional text generation.

2.1 DreamOn: Expanding beyond the fixed-size canvas

Mask-absorbing diffusion LMs (such as LLaDA [1]) generate by unmasking a canvas whose length is fixed in advance. This assumption fails in realistic settings where the ideal output length is unknown, for example code infilling, where the completion may be a single line or a whole function. DreamOn [2] directly addresses this: it augments the diffusion process with two length-control states that allow the model to autonomously *expand* or *contract* the output length based on its own predictions,

integrated with minimal modifications to the training objective and no architectural changes. Built on 7B-parameter diffusion backbones, DreamOn matches autoregressive state-of-the-art on code-infilling benchmarks and approaches the performance of an oracle that knows the ground-truth length, demonstrating that a diffusion model can be taught to negotiate its own length.

The length-growing primitive of DreamOn maps directly onto the **EXPAND** operation in this work. At inference, an EXPAND tag on a position replaces one mask slot with two fresh mask slots, increasing the canvas capacity; at training, spans of the clean target are compressed into a single EXPAND token so the model learns *when* growth is warranted. Alongside INSERT (content before a position) and DELETE (pruning), EXPAND completes a learned length-control triad: the model can lengthen, shorten, or keep each region of its output at every refinement step.

2.2 DiffusER: Edit-based diffusion with prompt-conditioned denoising

DiffusER [3] treats text generation not as unmasking but as *reconstruction through edits*. Its forward process corrupts a text by applying discrete edit operations – delete, insert, replace, and paste – and a Transformer is trained to reverse this corruption, producing text by iteratively refining an edit-based noisy representation. Because the operations mirror the Levenshtein edit distance, DiffusER naturally supports revision: the model does not merely fill blanks, it rewrites, removes, and reorders content. Crucially for conditional generation (for example machine translation), the source sentence acts as the conditioning context – the prompt – while the denoised target evolves under the same edit vocabulary; the model learns to drive a noisy target toward a correct translation by inspecting the source at every step. DiffusER demonstrates that an edit-based diffusion process can outperform autoregressive baselines on several seq2seq tasks while retaining the ability to refine existing text.

2.3 Combining both ideas in DLLM

This architecture unifies the two lines of work:

- From [3], it takes the notion of prompt-conditioned edit diffusion: a preserved prompt guides the denoising of a response canvas under Levenshtein-style operations (KEEP, DELETE, REPLACE, INSERT), applied iteratively.
- From [2], it takes the EXPAND operation, giving the model the ability to grow the response beyond its initial capacity rather than being confined to a fixed canvas.
- It re-frames the task as *prompt-to-output* (dialogue) generation: training pairs are consecutive dialogue turns, and the model must reverse a corruption applied only to the answer portion.
- It adds three mechanisms on top: (i) interleaved structural masks so prompt and response share a single position grammar; (ii) self-replacement supervision (`REPLACE` with the token itself as the generator target) that keeps the prompt pristine while training the generator densely; and (iii) a dual-head design – a tagger that decides *what edit to apply* and a generator, conditioned on the predicted tag embedding, that decides *what token to write*.

The result is a diffusion LM whose output length and content are both decisions taken iteratively under the guidance of a prompt, rather than a fixed-length fill-in-the-blank over a masked sequence.

3 Use Cases

3.1 Fast translation with expandable and contractable tokens

Machine translation is a natural fit for edit-based diffusion, and the dynamic-length machinery makes it especially attractive for translating between languages with very different verbosity. When translating from a terse language into a verbose one (or vice versa), the target length is unknown a priori and varies widely. DiffusER-style edit diffusion typically relies on corpus-derived length statistics and a fixed canvas budget, so a short German source may waste capacity that a long English rendering

requires. Here, the target canvas starts at a modest capacity and the model expands it (EXPAND, INSERT) precisely where the context demands more tokens – a rich clause, an explanatory phrase – and contracts it (DELETE) where the target is more compact than the slot layout. The length budget is thus renegotiated at every step, conditioned jointly on the source prompt and the partial translation already on the canvas, which is an advantage over fixed-budget edit-based translation models.

3.2 No static token budget at inference

Unlike mask-based diffusion models, generation here never commits to a final sequence length. The initial canvas is pure *capacity*, not a length specification: the number of slot pairs allocated at step zero is only an upper bound on the response. Each refinement step mutates the canvas – tokens are deleted, replaced, inserted, expanded – and convergence is declared only when the tagger is satisfied that every response position should be kept. The same prompt can therefore yield a one-sentence answer (most capacity pruned via DELETE) or a long elaboration (capacity grown via EXPAND and INSERT), and the decision is made per step, per position, using the full bidirectional context. This is precisely the flexibility that fixed-length masked diffusion models such as LLaDA lack.

3.3 Parallel attention advantages over autoregressive models

The backbone is a bidirectional encoder (RoBERTa), so every position attends to the entire canvas – the full prompt and all partially generated content – at every iteration. This yields three practical advantages over autoregressive decoding:

- *Parallel decode*: all positions of the canvas are predicted in a single forward pass, and the whole canvas is updated per iteration; wall-clock cost is roughly (iterations) $\times$ (one forward pass), independent of response length.
- *Bidirectional context*: tokens are never constrained to condition only on the left; the model can let the end of a sentence influence its beginning, which is impossible for causal attention.
- *No compounding errors*: early drafts can be revised by later iterations (REPLACE, DELETE), instead of being frozen and propagated forward.

3.4 A small-wiki chatbot

The prompt-to-output formulation makes the model data-efficient in a way autocomplete models are not. Every training position receives supervision – a tag target and, where needed, a token target – so even a small corpus yields dense gradient signal. Training on dialogue-turn pairs extracted from a small wiki-like corpus (e.g. the tiny Shakespeare corpus used in the implementation) produces a chatbot that answers directly: the model is never asked to continue the prompt token-by-token, but to reverse a corruption applied to the answer. For small, specialized domains, a domain wiki can be turned into turn pairs with simple speaker-extraction heuristics, and a 164M-parameter bidirectional backbone suffices to train a competent, fast, non-autoregressive answer generator.

3.5 Difference from LLaDA and similar mask-based diffusion LMs

Aspect	**LLaDA / MDLM**	**DLLM**
Length	Fixed L chosen up front; monotonic unmasking	Dynamic: capacity canvas; length decided by KEEP/ DELETE/INSERT/EXPAND per step
Operations	Mask $\rightarrow$ unmask only	KEEP, DELETE, REPLACE, INSERT, EXPAND

Revision	Confidence-based re-masking heuristic	Native: REPLACE and DELETE are trained operations
Heads	Single token-prediction head	Dual: Tagger (ops) + tag-conditioned Generator (tokens)
Prompt	Unmasked fixed prefix	Interleaved structural masks + REPLACE-self reconstruction
Theory	Continuous-time discrete diffusion (ELBO, schedules)	Empirical edit-path supervision (t-noise curriculum)

The central advantage over LLaDA and its relatives is the absence of a pre-committed output length and the presence of real revision operations: LLaDA decides how long its answer will be before it starts, and can only refine by unmasking; DLLM negotiates length and content jointly, iteratively. The costs are a larger tag-decision surface (operation-class imbalance, mitigated here by class-weighted tag loss and a skewed corruption schedule) and the overhead of structural mask tokens in the canvas.

4 Implementation

This section walks through the implementation end to end. The system is organized as a Python package (`dllm`) with training and generation scripts, and a YAML configuration. All components are available in the repository: `tokenizer.py`, `corruptor.py`, `dataset.py`, `model.py`, `trainer.py`, `inference.py`, `utils.py`, and the entry points `scripts/train.py` and `scripts/generate.py`.

4.1 Extended tokenizer

The tokenizer wraps a standard RoBERTa tokenizer (`roberta-base`) and appends six edit-operation tokens, raising the vocabulary from 50,265 to 50,271:

Token	**ID**	**Semantics**
`<KEEP>`	50265	Token is correct; no change
`<DELETE>`	50266	Token is spurious; remove it
`<REPLACE>`	50267	Substitute the position with a new vocabulary token
`<INSERT>`	50268	Insert a new token before this position
`<EXPAND>`	50269	Grow this position into two mask placeholders
`<MASK>`	50270	Placeholder whose content must be generated

The tokenizer caches the IDs of all six tokens, the original vocabulary size (used for generator weight tying), and the base special tokens.

4.2 Data preparation

Training data is loaded from a text corpus (by default tiny Shakespeare). In `prompt_response` mode, consecutive dialogue turns are extracted into `(prompt, response)` pairs using speaker-heading heuristics: a line is treated as a speaker turn if it ends in a colon and is short and/or uppercase; fallback sentence chunking produces pairs when no speakers are found. Prompts and responses are tokenized to at most 48 tokens each.

4.3 Forward corruption: the training-time diffusion process

The corruptor builds the training canvas. For each sample it draws a noise level t from a distribution skewed toward full masking,

$$t = 1 - U(0,1)^s,\ s = 2$$

so that a substantial fraction of samples are near-fully-masked – matching the all-mask canvas that inference starts from. The canvas is then assembled:

```
[BOS] <MASK> p1 <MASK> p2 ... <MASK> | [r1 iMASK] [r2 iMASK] ... [rN iMASK] |
[EOS]
       prompt section (pristine)         response section (corrupted)
```

- **Prompt section**: each prompt token is preceded by an interleaved mask tagged `KEEP`; the prompt token itself is tagged `REPLACE` with itself as the generator target (self-replacement). The prompt is thus preserved verbatim while the generator receives dense training signal.
- **Response section**: a canvas of N slot pairs, where N is sampled per sample as response-length plus a random tail pad of 0-6 slots (capped at 32), so the `DELETE` class stays rare and the canvas size varies across samples. For each slot, with r_i the clean token and roll $\sim U(0,1)$:
  - $\text{roll} > t$: the clean token r_i is placed and tagged `KEEP` (verification);
  - $\text{roll} < t \cdot \text{mask\_ratio}$: the slot holds a `MASK`, tagged `REPLACE` (target r_i) or, with small probability, `EXPAND`;
  - otherwise: the slot holds a random noise token, tagged `REPLACE` (target r_i) (in-place refinement).

  The structural interleaving mask after each slot is tagged `INSERT` (target r_{i+1}) with probability `insert_prob`, else `KEEP`. Tail slots beyond the response hold `MASK` pairs tagged `DELETE`.
- **Trailing EOS** closes the canvas, tagged `KEEP`.

A `pos_to_clean` map records the exact clean token for every `REPLACE`/`INSERT` position, so generator labels are exact and heuristic-free. A separate, simpler corruption path (`corrupt`), which applies random replace/delete/insert/expand/mask noise and computes edit labels by Levenshtein alignment over a private-use-unicode encoding of the token IDs, supports an unconditional (non-dialogue) training mode.

4.4 Dataset and collation

`__getitem__` performs tokenization and corruption dynamically, so every epoch presents different corruptions of the same underlying text. Each sample returns `noisy_ids`, `attention_mask`, `tag_labels` (edit tags, padded with `-100`), `gen_labels` (token targets at generation positions, padded with `-100`), and `gen_mask` (boolean mask of active generation positions). The collate function filters invalid samples and stacks the batch.

4.5 Model

The model is a dual-head bidirectional transformer:

- **Backbone**: `roberta-base` (all 12 layers attend bidirectionally), with embeddings resized to 50,271 tokens.
- **Tagger head**: hidden state → Linear → GELU → LayerNorm → Linear(5) – per-position logits over the five edit operations.

- **Generator head**: hidden state plus a learned tag embedding (conditioned on the target tag during training, the predicted tag during inference) → Linear → GELU → LayerNorm → Linear(50,265), with the final projection weight-tied to the backbone input embeddings for the original vocabulary.

The joint loss is

$$L = L_{\text{tag}} + L_{\text{gen}}$$

where L_{tag} is a class-weighted cross-entropy over edit tags (weights `[0.7, 0.3, 1.0, 2.0, 4.0]` for KEEP/DELETE/REPLACE/INSERT/EXPAND, countering the majority-class bias), and L_{gen} is a cross-entropy evaluated only at positions marked for generation (REPLACE/INSERT), with an ignore index everywhere else. The model has roughly 164M parameters.

4.6 Training

Training uses AdamW with weight decay applied only to matrix weights (biases and LayerNorm are excluded), a linear warmup of 500 steps followed by linear decay to `max_steps` (50,000), gradient accumulation over 4 micro-batches, and gradient-norm clipping at 1.0. A validation loop evaluates tag, generator, and length loss periodically; only the best-validation checkpoint is stored (no periodic checkpoints), saving model, optimizer, scheduler, and step counters. Two empirically motivated design choices are worth noting: the corruption schedule is skewed toward full masking and `mask_ratio` is high (0.8) so the model trains on canvases that resemble the all-mask inference start, and the `DELETE` tail-padding is capped at six slots with class weighting on the tag loss – together these prevent the tagger from collapsing to a degenerate "delete everything" policy on fully masked canvases.

4.7 Inference: iterative refinement

Generation starts by building the same canvas structure as training, with the user-chosen `target_length` slot pairs. Each position carries a type label (`bos`, `prompt_imask`, `prompt_tok`, `response_slot`, `response_imask`, and the derived `response_filled` / `response_inserted` types) so rules survive length changes:

1. Forward pass in evaluation mode; the tagger produces per-position operation logits (argmax decides the edit), and the generator produces vocabulary logits.
2. Tokens are sampled at generation positions with temperature 1.0, top-k = 50, top-p = 0.9.
3. Edits are executed with type propagation: `KEEP` keeps the position; `DELETE` removes it; `REPLACE` substitutes the sampled token (a mask slot becomes `response_filled`); `INSERT` places the sampled token before the position, leaving the mask as an anchor; `EXPAND` turns one mask into two new slots. Structural positions (`bos`, `eos`, `prompt_imask`) are hard-locked to `KEEP`, and a `KEEP` prediction on a `prompt_tok` is forced to `REPLACE`-self, so the prompt can never be modified.
4. The loop repeats (maximum 20 iterations) and converges when all response positions predict `KEEP` or the canvas stops changing; remaining masks are stripped during decoding.

Optional trajectory logging records the raw canvas, the cleaned canvas, the extracted response, and tag statistics at every step.

4.8 Configuration and scripts

A single YAML file (`configs/default.yaml`) controls the model, tokenizer, data, corruption, training, and inference settings. `scripts/train.py` wires the components together, prints a sanity check of one batch, and launches the trainer; `scripts/generate.py` loads a checkpoint and performs the iterative refinement described above, optionally displaying the full denoising trajectory.

References

References

[1] S. Nie *et al.*, "LLaDA: Large Language Diffusion with mAsking." [Online]. Available: https://arxiv.org/abs/2502.09992

[2] Z. Wu *et al.*, "DreamOn: Diffusion Language Models For Code Infilling Beyond Fixed-size Canvas." [Online]. Available: https://arxiv.org/abs/2602.01326

[3] M. Reid, V. J. Hellendoorn, and G. Neubig, "DiffusER: Discrete Diffusion via Edit-based Reconstruction." [Online]. Available: https://arxiv.org/abs/2210.16886